

RetailFlow

Plan de préparation Orisha • Vendredi → Mardi

Objectif : arriver à l'entretien avec un projet concret couvrant la stack du poste et une préparation d'entretien structurée.

Stack cible : C# / ASP.NET Core / REST / EF Core / PostgreSQL / React / TypeScript / Azure / CI-CD / tests / SaaS multi-tenant.

1. Backlog global

Priorité	Ticket	Livrable / Definition of Done
P0	PROJ-001 Initialisation	Repo Git + solution .NET + React + README + lancement local
P0	PROJ-002 Domaine Retail	Product, Tenant, Store, Inventory, StockMovement, Sale/SaleLine
P0	PROJ-003 API	CRUD produits + stock + ventes + validation + erreurs HTTP
P0	PROJ-004 PostgreSQL	EF Core migrations + relations + indexes + données seed
P0	PROJ-005 Métier	Stock in/out + PUMP + marge + règles transactionnelles
P0	PROJ-006 Multi-tenancy	TenantId + isolation des données + tests d'accès
P0	PROJ-007 Frontend	React/TS dashboard produits, stock, mouvements, ventes
P0	PROJ-008 Tests	Unit tests métier + tests API critiques
P1	PROJ-009 CI/CD	Pipeline build/test + déploiement Azure
P1	PROJ-010 Observabilité	Logs structurés + health check + Application Insights si temps
P1	PROJ-011 Présentation	Architecture diagram + README + démo scriptée
P0	INT-001 Pitch	Pitch FR + EN de 60 secondes
P0	INT-002 C#/NET	DI, async, LINQ, IQueryable/IEnumerable, SOLID, HTTP
P0	INT-003 SQL/Architecture	transactions, indexes, multi-tenancy, scalabilité
P0	INT-004 Retail	PUMP, marge, TVA, stock, concurrence
P0	INT-005 Orisha	produits, BU Commerce, squad, questions à poser
P0	INT-006 Simulation	2 simulations complètes + correction

VENDREDI — Fondations + backend

[PROJ-001] Créer le repo et l'arborescence : /src/api, /src/web, /tests, /docs. Initialiser ASP.NET Core Web API et React TypeScript. Ajouter .gitignore, README et configuration locale.

[PROJ-002] Modéliser Tenant, Store, Product, Inventory, StockMovement, Sale et SaleLine. Garder le domaine simple. Pas de microservices.

[PROJ-004] Installer EF Core PostgreSQL. Créer DbContext, relations, migrations et seed minimal.

[PROJ-003] Créer GET/POST/PUT/DELETE Product et GET Inventory. Ajouter DTOs, validation et codes HTTP corrects.

[INT-002] Réviser C#/.NET : DI, async/await, Task, LINQ, DTO, middleware, configuration, REST, 400/401/403/404/409/500.

Definition of Done : Fin de journée : API démarre, DB démarre, migration fonctionne, au moins 2 endpoints testés via Swagger.

SAMEDI — Métier + frontend

[PROJ-005] Implémenter réception stock, sortie stock, vente, PUMP et marge. Utiliser decimal/numeric pour les montants.

[PROJ-006] Ajouter TenantId aux données métier. Déterminer le tenant à partir d'un contexte simple pour la démo. Ne jamais accepter aveuglément un TenantId venant du client.

[PROJ-007] Créer layout React + navigation + pages Products, Inventory, Movements, Sales. API client typé.

[INT-003] Réviser SQL : JOIN, index, transaction, concurrence, EXPLAIN ANALYZE. Préparer une explication du choix multi-tenant.

[INT-004] Préparer PUMP, marge et TVA avec exemples chiffrés.

Definition of Done : Fin de journée : parcours complet produit → stock → vente → marge visible dans le navigateur.

DIMANCHE — Qualité + architecture

[PROJ-008] Tester les règles critiques : PUMP, stock insuffisant, vente, marge, isolation tenant.

[PROJ-010] Ajouter health check, logs utiles et gestion centralisée des erreurs. Application Insights seulement si le cœur est stable.

[PROJ-011] Créer diagramme : React → API → application/domain → EF Core → PostgreSQL. Documenter les choix.

[INT-001] Écrire et répéter pitch FR/EN.

[INT-005] Réviser Orisha Commerce, SaaS, Retail et préparer 5 questions techniques/produit.

Definition of Done : Fin de journée : projet démontrable et explicable sans lire le code.

LUNDI — Azure + CI/CD + durcissement

[PROJ-009] Créer pipeline : restore → build → test → publish. Déployer l'API sur Azure. Déployer le frontend si possible.

[PROJ-004] Vérifier configuration DB de production, secrets hors Git, migrations contrôlées.

[PROJ-006] Ajouter tests d'isolation tenant et vérifier qu'un tenant ne peut pas lire/modifier les données d'un autre.

[PROJ-007] Polir UX minimale : loading, erreurs, validation, tableau stock.

[INT-006] Simulation technique 1 : C#/.NET/SQL/architecture. Noter chaque trou de connaissance.

Definition of Done : Fin de journée : URL ou environnement de démo fonctionnel, pipeline vert, aucun secret commité.

MARDI — Répétition finale + entretien

[PROJ-011] Préparer une démo de 7–10 min : problème → architecture → scénario métier → code clé → tests → déploiement → limites.

[INT-006] Simulation 2 : 45–60 min, puis simulation RH 20 min. Réponses courtes, concrètes, avec exemples personnels.

[INT-002] Réviser les 20 questions techniques prioritaires ci-dessous.

[INT-005] Réviser les questions à poser à l'équipe et le pitch Orisha.

Definition of Done : Objectif : ne plus apprendre de nouvelle technologie. Stabiliser, répéter, dormir. Le cerveau humain a des limites ridicules mais documentées.

2. Guide step-by-step du projet

1. Étape 1 — Bootstrap

Installer/valider .NET SDK, Node.js, PostgreSQL et Git. Créer le repo. Le premier commit doit seulement prouver que le projet démarre.

2. Étape 2 — API ASP.NET Core

Créer Web API. Ajouter Swagger/OpenAPI. Créer dossiers Controllers, Application/Services, Domain, Infrastructure, DTOs. Ajouter DI.

3. Étape 3 — Domaine

Créer les entités et les règles métier. Éviter de mettre la logique de PUMP ou de vente directement dans un Controller.

4. Étape 4 — EF Core + PostgreSQL

Installer Npgsql/EF Core. Définir DbContext. Configurer clés, relations, contraintes et indexes. Faire migration puis seed.

5. Étape 5 — Produits

Créer ProductService puis ProductController. DTO de création/modification séparés du modèle DB. Tester avec Swagger.

6. Étape 6 — Stock

StockMovement est la trace. Inventory représente l'état courant. Chaque opération doit maintenir la cohérence entre les deux.

7. Étape 7 — PUMP

Pour une réception : nouveau coût moyen = (ancienne valeur + nouvelle valeur) / nouvelle quantité. Utiliser decimal. Documenter les arrondis.

8. Étape 8 — Vente

Vérifier disponibilité, calculer coût/marge, créer Sale + SaleLine + mouvement OUT dans une transaction.

9. Étape 9 — Multi-tenant

Chaque donnée métier possède TenantId. Le service filtre systématiquement par tenant. Préparer un test A ne voit pas B.

10. Étape 10 — React

Créer composants simples. API client centralisé. Types Product, Inventory, Sale. Gérer loading/error/empty states.

11. Étape 11 — Tests

Tester les règles métier indépendamment de la DB quand possible. Ajouter quelques tests d'intégration API pour les scénarios critiques.

12. Étape 12 — CI/CD

Pipeline automatique build/test. Puis publication. Les secrets restent dans les variables/secrets Azure/GitHub, jamais dans Git.

13. Étape 13 — Déploiement

API Azure + PostgreSQL managé. Configurer connection string/variables. Vérifier health endpoint et logs.

14. Étape 14 — Démo

Préparer un scénario unique : créer produit → réception 100 à 10€ → réception 50 à 14€ → vendre 30 à 20€ → afficher PUMP et marge → montrer isolation tenant → montrer tests/pipeline.

3. Questions techniques prioritaires

- Pourquoi ASP.NET Core et pourquoi une couche Service entre Controller et persistance ?
- DI : qu'est-ce que l'injection de dépendances et pourquoi l'utiliser ?
- IEnumerable vs IQueryable : où la requête s'exécute-t-elle ?
- Pourquoi async/await pour les appels DB/API ?
- DTO vs Entity : pourquoi les séparer ?
- Comment gérer les erreurs API de façon cohérente ?
- 401 vs 403 ?
- Quand utiliser une transaction ?
- Comment éviter une vente simultanée de la même dernière unité ?
- Quel index créer sur StockMovement et pourquoi ?
- Comment investiguer une requête PostgreSQL lente ?
- Pourquoi decimal plutôt que double pour les prix ?
- Comment calculer le PUMP ?
- Comment concevoir un SaaS multi-tenant ?
- Comment garantir l'isolation entre tenants ?
- Shared schema vs schema/database par tenant : compromis ?
- Comment versionner une API sans casser les clients ?
- Comment tester une règle métier complexe ?
- Que mettez-vous dans CI/CD avant déploiement ?
- Que surveiller en production : erreurs, latence, DB, disponibilité, logs ?

4. Script de présentation — 8 minutes

0:00–0:45 — Contexte : « J'ai choisi un cas Retail parce qu'il reprend les problématiques de l'offre : produits, stock, ventes, marge et multi-tenancy. »

0:45–2:00 — Architecture : React/TypeScript → ASP.NET Core REST → services métier → EF Core → PostgreSQL. Expliquer les responsabilités.

2:00–4:00 — Démo métier : réception → PUMP → vente → marge. Montrer la cohérence transactionnelle.

4:00–5:00 — Multi-tenant : montrer deux tenants et le test d'isolation.

5:00–6:00 — Qualité : tests critiques, validation, erreurs, health check.

6:00–7:00 — CI/CD et Azure : pipeline build/test/deploy et configuration des secrets.

7:00–8:00 — Limites : authentification réelle, RBAC, concurrence avancée, observabilité complète, scaling. Expliquer ce que tu ferais ensuite.

5. Règles de travail pendant les 5 jours

- Priorité P0 avant tout P1. Une feature incomplète vaut moins qu'un parcours complet qui fonctionne.
- Pas de microservices. Un modular monolith est volontairement suffisant pour ce projet de démonstration.
- Pas de sur-engineering. Chaque abstraction doit avoir une raison que tu peux expliquer.
- Chaque session de code se termine par un commit clair.
- Ne jamais copier du code généré sans comprendre les classes, dépendances et flux principaux.

- Quand une techno bloque plus de 30 minutes, réduire le scope et avancer. Le projet sert l'entretien, pas l'inverse.
- Le lundi soir, aucune nouvelle grosse feature. Le mardi est réservé à la répétition.

6. Checklist finale

- API démarre localement
- PostgreSQL démarre et migration fonctionne
- CRUD Product fonctionne
- Stock IN/OUT fonctionne
- PUMP fonctionne avec decimal
- Vente + marge fonctionne
- Transaction utilisée pour une opération critique
- Multi-tenancy démontrable
- React affiche produits et stock
- Tests critiques passent
- Pipeline build/test passe
- Déploiement accessible
- README explique architecture et lancement
- Pitch FR mémorisé
- Pitch EN mémorisé
- 5 histoires professionnelles préparées
- 5 questions Orisha préparées
- Démo chronométrée à 8–10 min